

ABSTRACT

This project proposes a portable, low-power IoT node for real-time environmental health monitoring using the STM32 B-L4S5I-IOT01A Discovery Kit. The system is designed to deliver immediate and actionable air-quality feedback to users. It performs multi-sensor data fusion by combining on-board environmental sensors (temperature and humidity) with external sensors, including the PMS5003 for PM2.5 measurement via UART and the SGP30 for TVOC sensing via I2C.

The STM32's floating-point unit (FPU) is utilized to compute a localized Air Quality Index (AQI), which directly controls the system's output. Real-time AQI values are presented on an LCD display, while air-quality severity is indicated using blinking the LED. To ensure portability and energy efficiency, the system employs a low-power design, periodically waking from Stop Mode using the RTC for sensor sampling. While MQTT-based cloud connectivity is planned for future expansion, the current prototype successfully demonstrates robust real-time sensing, processing, and local visualization platform focused on enhancing user safety.

Team Name: Chipspire

Team Leader: Karra Sai Siddartha

Team Members:

1. Karra Sai Siddartha (UH ID: 2443188)
2. Narendhar Puttinti (UH ID: 2454090)
3. Manaswini Gundapaneni (UH ID: 2443790)
4. Shaik Irshad Ahmed (UH ID: 2443624)
5. Mukesh Musham (UH ID: 2430886)

IoT-Enabled Portable Multi-Hazard Air Quality Monitoring system

I. Introduction

The project addresses the pressing public health challenge due to localized air pollution, especially from harmful agents like PM_{2.5} and volatile organic compounds. Traditional systems for air quality monitoring often use fixed stations providing generalized data for big areas, missing micro-environment variability that affects human health directly. "IoT-Enabled Portable Multi-Hazard Air Quality Monitoring system" was conceived to overcome these limitations by offering a portable, real-time monitoring device bringing environmental data directly to the user.

This system will utilize the potential of a B-L4S5I-IOT01A Discovery board that embeds a high-performance STM32L4S5 microcontroller that features an ARM Cortex-M4 core. There are onboard sensors and expansion I/O pins available in this device. Such a selection provides a low-power, high-performance backbone that suits IoT applications. The architecture is based on industry-standard protocols: I2C for sensor communication, UART for specialized PM sensor input, and SPI for the high-speed LCD display interface. Data acquisition and management are done through the FreeRTOS real-time operating system that permits concurrent, clash-free, stable running of the multiple sensor tasks.

The key objectives of the project consisted of seamless aggregation of various sensor inputs, data integrity guaranteed via proper software design patterns (including mutexes), processing of raw data into useful metrics, and clear, immediate visualization of environmental conditions on a local display. A system providing this reliable base to monitor these parameters is able to give reasons for health awareness and proactive measures against environmental hazards in a wide range of settings.

II. System Overview

IoT-Enabled Portable Multi-Hazard Air Quality Monitoring system is an end-to-end IoT-based health solution powered by the STM32 B-L4S5I-IOT01A development board. The critical environmental parameters of temperature, humidity, pressure, and particular hazards about air quality are constantly monitored with the help of onboard integrated sensors-HTS221 and LPS22HB-and external modules like SGP30 and PMS5003. The data so acquired is processed in a FreeRTOS environment and sent to the local ST7735 LCD display for real-time local monitoring and immediate generation of alerts.

The core functionalities of the system are: runtime environment monitoring for all five parameters and data processing with a dedicated library of SGP30 algorithms. Further developments will include onboard Wi-Fi modules to connect to the cloud and external body temperature sensors. The goal is to contribute to environmental safety by continuous tracking and logging of data. The

entire implementation is designed with scalability and modularity, supporting both personal use and clinical healthcare monitoring applications.

III. System Architecture

The architecture of the system is a multi-layered design focused on reliable, low-power data acquisition from diverse environmental sensors and local display of processed information. It is logically divided into hardware and software components interacting via standardized communication protocols.

I. Hardware Architecture

The hardware architecture centers around the STM32 B-L4S5I-IOT01A microcontroller board, which provides a powerful ARM Cortex-M4 core, integrated sensors, and versatile expansion headers like Pmod and ST Morpho for interfacing external components.

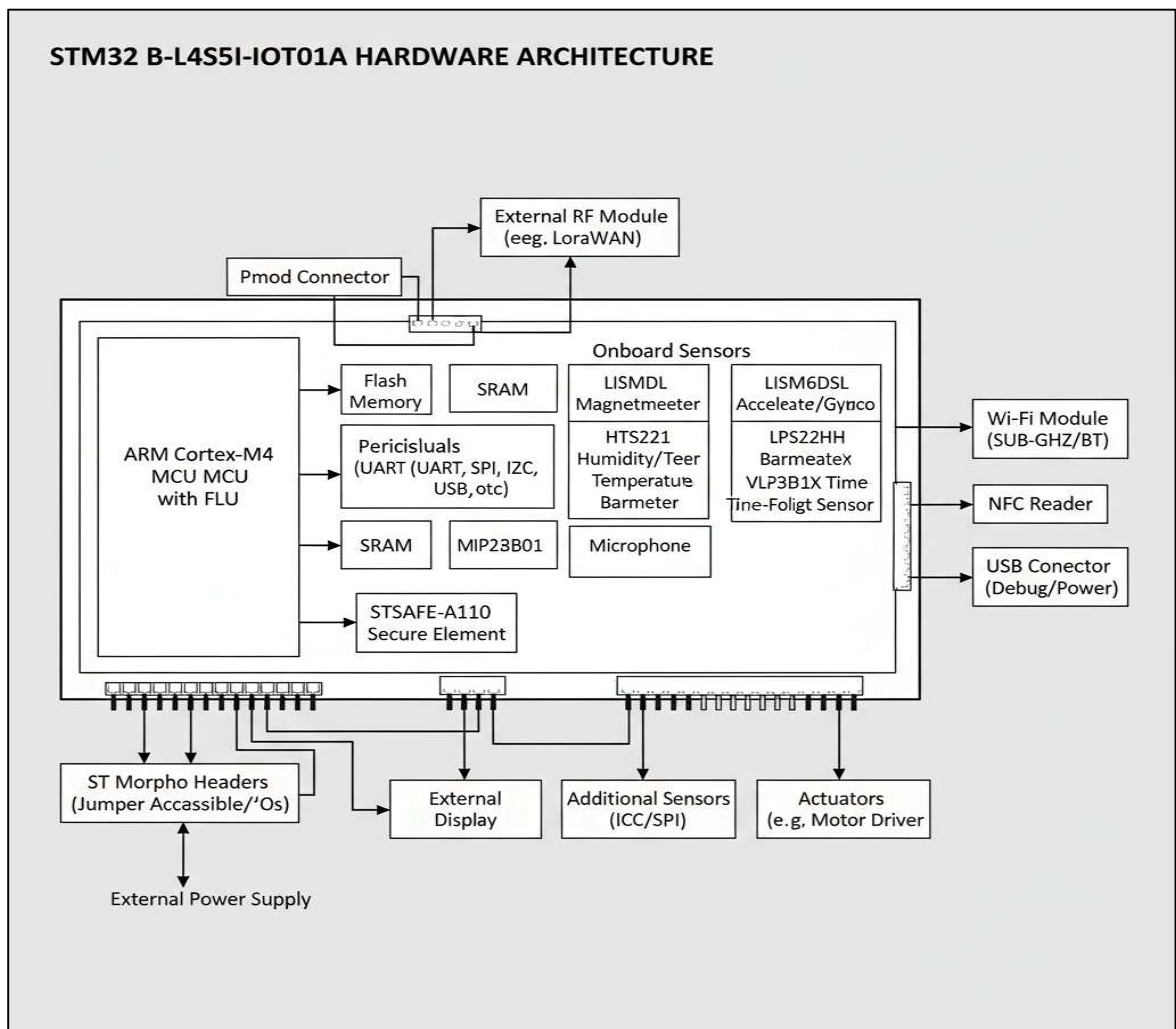


Figure 1: System Architecture

1.1 Core Microcontroller Specifications

Microcontroller: ST STM32L4 Series (ARM Cortex-M4 core)
Operating Frequency: 80 MHz
Flash Memory: 1 MBRAM: 128 KB
Power Supply: Primarily 3.3V (5V supplied to the PMS5003 sensor)

1.2 Integrated and External Sensors

Sensor Type	Model	Parameter Monitored	Interface	Configuration Details
Temperature / Humidity	HTS221	Temperature (°C), Humidity (%)	I2C	2 Address 0x5F (0xBE/0xBF)
Pressure Sensor	LPS22HB	Pressure (hPa)	I2C	2 Address 0x5D (0xBA/0xBB)
VOC Sensor (External)	SGP30	VOC Index (0-500)	I2C	1 Address 0x42 (0x84/0x85), uses PB8/PB9 pins
PM2.5 Sensor (External)	PMS5003	PM2.5 (µg/m³)	UART	49600 baud rate, uses PA0/PA1 pins
Display	ST7735	Local data visualization	SPI	Uses standard SPI interface pins

1.3 Communication Interfaces

The system integrates various serial interfaces to support efficient and reliable data transfer between components: I²C, UART, and SPI.

System Architecture Block Diagram Description diagram showing the system architecture should depict the central STM32L4 MCU. Inputs on the left flow into the MCU, that is, I2C Sensors (HTS221, LPS22HB, SGP40) and the UART Sensor (PMS5003), flow into the MCU on the left. Output flows to the right, that is, the SPI LCD Display. Internal connections within the MCU box show the FreeRTOS middleware, the I2C/UART/SPI peripherals, and shared data memory protected by the Mutex.

II. Software Architecture

The system uses a layered software architecture, implemented on top of the FreeRTOS real-time operating system, to ensure truly parallel data acquisition, processing, and display.

2.1 Firmware Layer

Development Environment: STM32CubeIDE was used for development, along with STM32CubeMX for initial peripheral configuration.

Middleware: FreeRTOS manages concurrent tasks, using tasks, mutexes [sensorDataMutex], and queues for inter-task communication and synchronization.

Key Functions: Sensor initialization routines [Init_HTS221, SGP40_Init_And_Algorithm_Start], data reading helpers [Read_HTS221_Temperature, PMS5003_ParseData], and task loop functions [HumidPressRead, LCDDisplay].

2.2 Application Layer

Algorithm: The key algorithm comes from the external Sensirion Gas Index Algorithm library - the only part able to process a raw data signal from the SGP30 into a standard, comparable VOC Index value [0-500].

Interface: The main user interface of the application is the LCD Display task, reading global variables safely through that mutex and displaying it in an easy-to-read format.

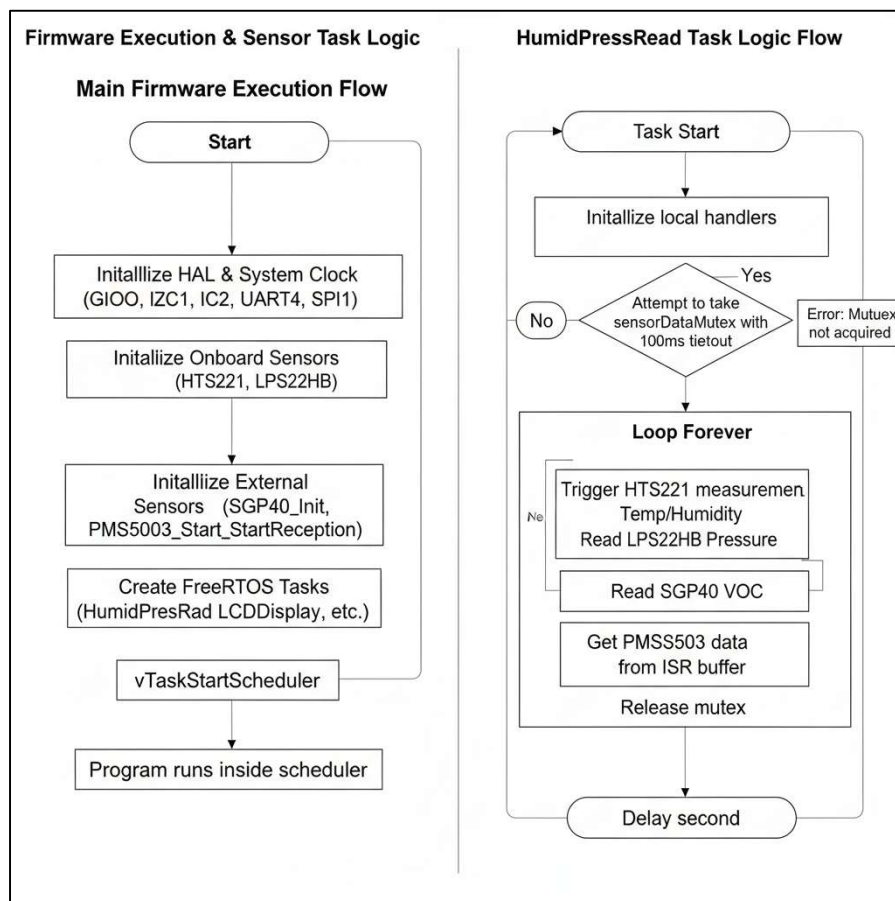


Figure 2: Block Diagram

IV. Project Objectives:

Primary Objective: Develop a real-time, autonomous embedded system using an STM32 microcontroller and FreeRTOS to monitor and display environmental air quality.

Key Objectives:

Multi-sensor integration: Interface HTS221, LPS22HB, SGP30, and PMS5003 sensors using I2C and UART protocols.

RTOS: Utilize the FreeRTOS to manage tasks running concurrently using mutexes that ensure integrity of data and non-blocking delays.

Data processing: transform the raw sensor signals into consistent units (e.g., ppm, $\mu\text{g}/\text{m}^3$) with the datasheet formula.

Air Quality Interpretation: Apply logic to classify overall air quality as "UNHEALTHY", "MODERATE", or "GOOD" based on standardized thresholds according to Air Quality.

User Interface: Real-time data and general air quality status is displayed on the ST7735 SPI LCD.

Reliability: implement error checking and interrupt-driven data acquisition via UART callbacks.

V. Embedded System Development

The project "Air Quality Monitor" itself is a very good example of Embedded Systems Development. It includes the development of a dedicated, resource-constrained system, which reads physical environmental data, processes it, and presents it to the user through a user interface.

Key embedded system aspects of our project include the

Core Microcontroller: STM32L4S5 is the brain of the system, running the special-purpose firmware developed particularly for this single purpose.

4.1. Hardware Interfacing: It needs to have low-level programming to communicate directly with external peripherals using certain protocols:

I2C for HTS221, LPS22HB, and SGP30 sensors.

UART for the PMS5003 sensor.

SPI for the ST7735 LCD display.

4.2. Real-Time Operation: Utilizing the FreeRTOS qualified this as a real-time embedded system; it will also ensure real-time data acquisition and updating of the display using task management, interrupts, and synchronization primitives such as Mutexes.

4.3. Resource Management: The software should efficiently manage the GPIOs of the microcontroller, the memory (RAM) for variables such as sensor data and task stacks, and the processing power for the handling of sensors, data calculation, and display refresh together.

5.4. Firmware Emphasis: The development process fully relies on the development of optimized firmware, in C language, highly integrated with your specific board configuration created through STM32CubeIDE.

5.5. Circuit Design:

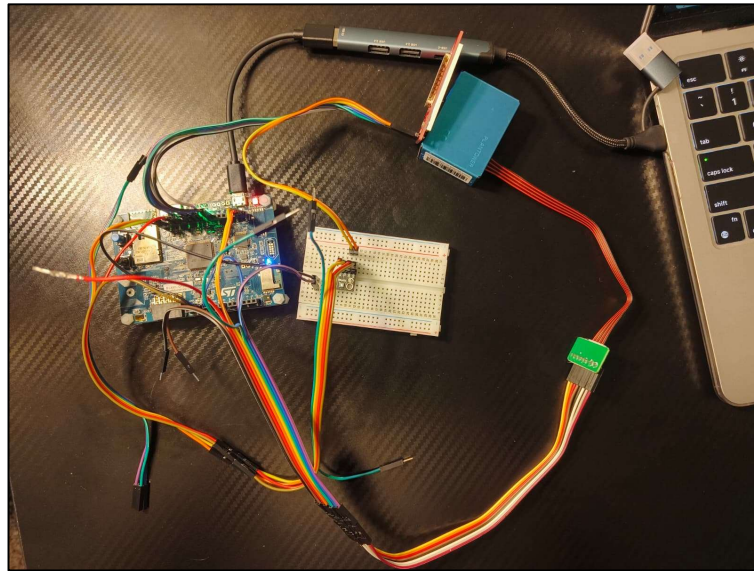


Figure 3: Circuit Design

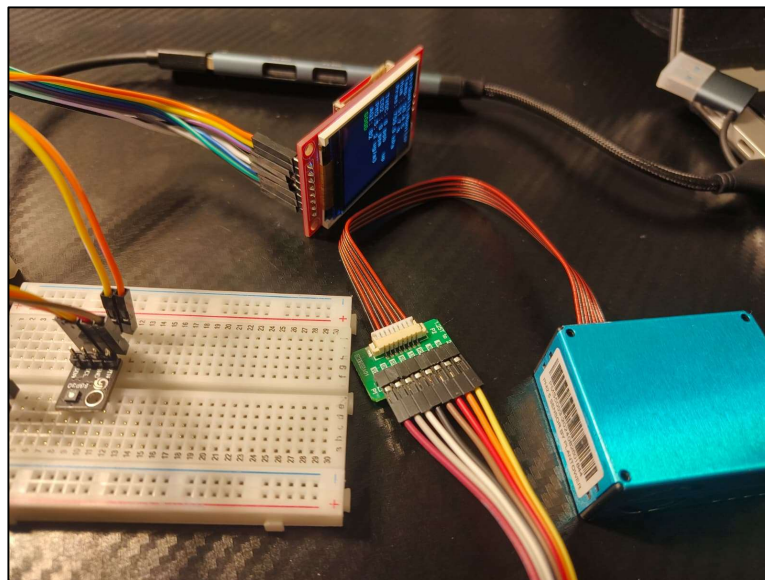


Figure 4: PM2.5 Sensor & VOC Sensor

5.6. Codes:

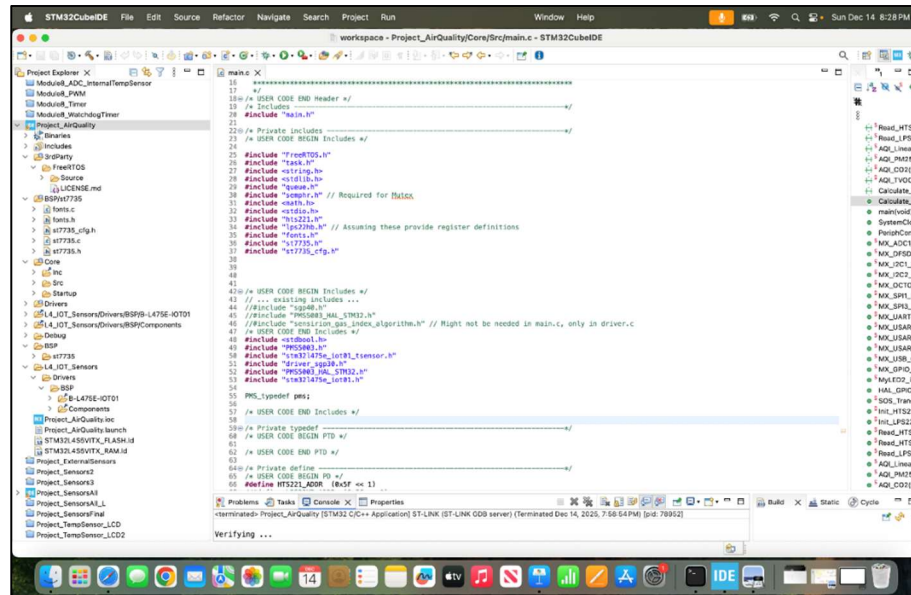


Figure 5:

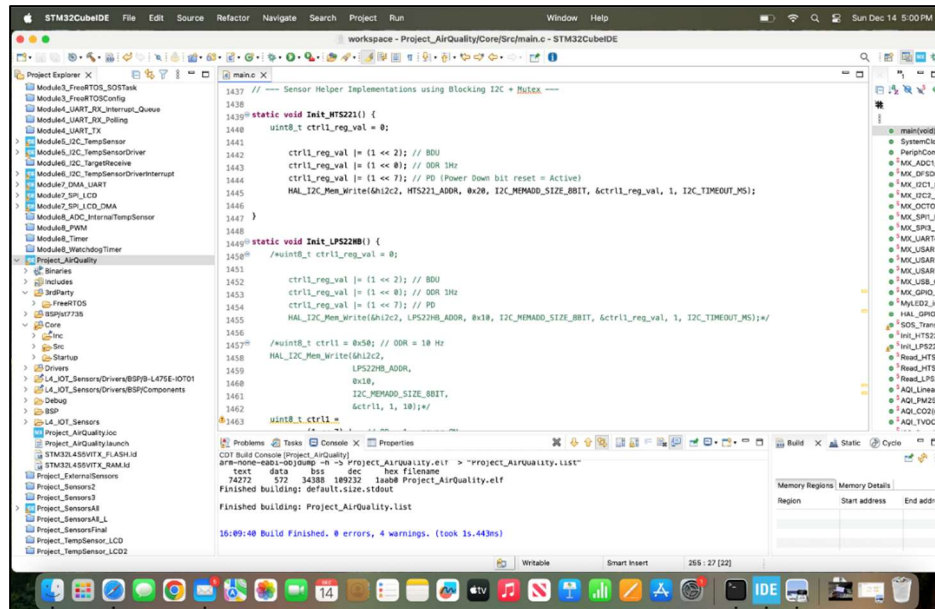


Figure 6:



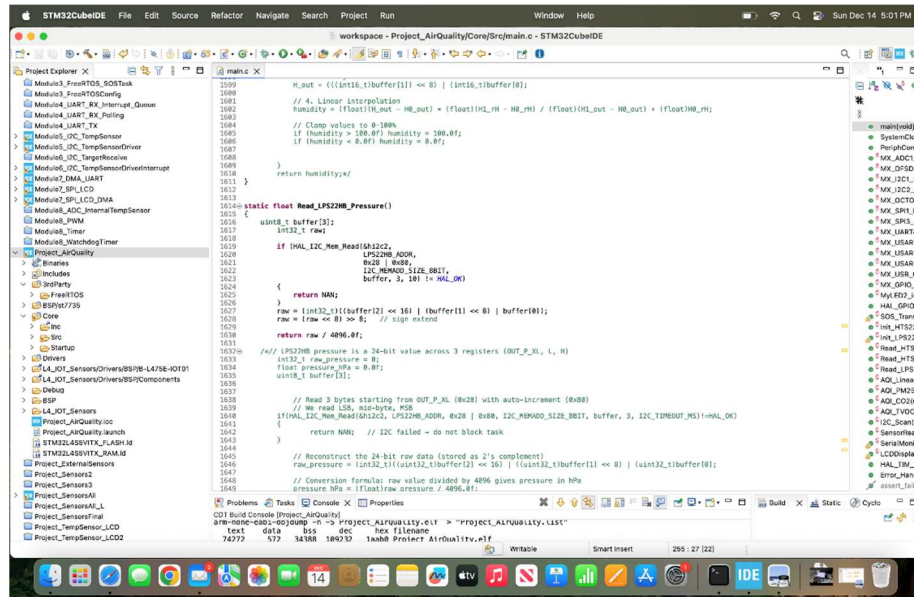


Figure 9:

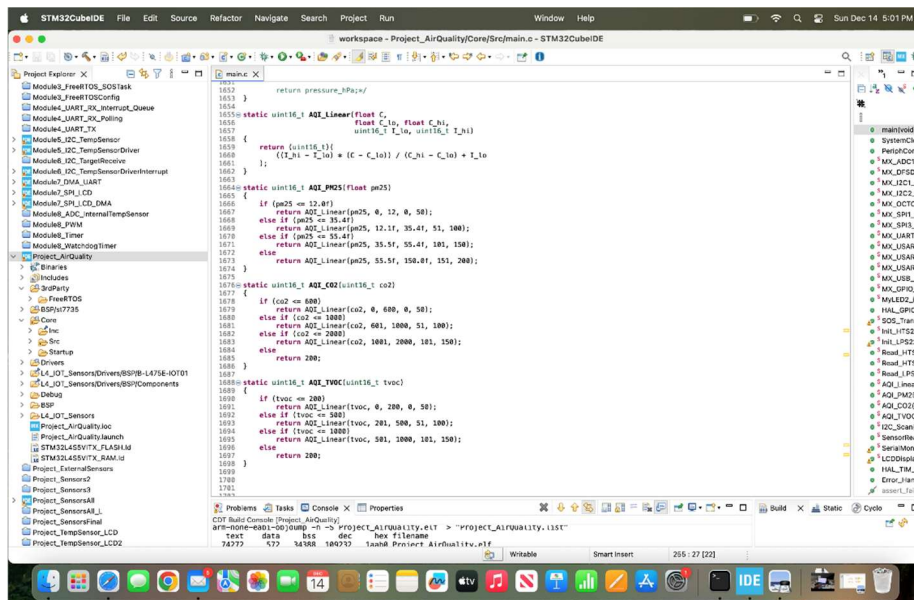


Figure 10:

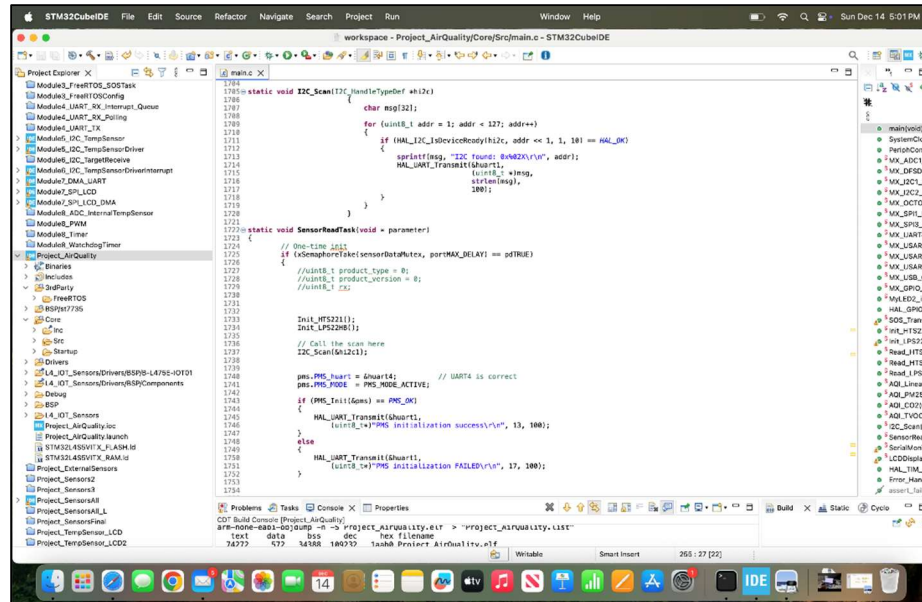


Figure 11:

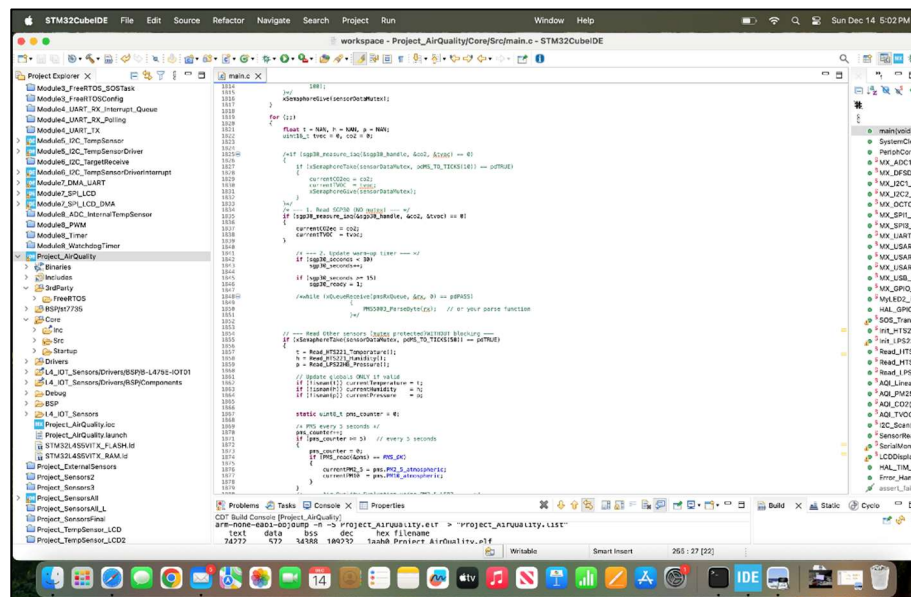


Figure 12:

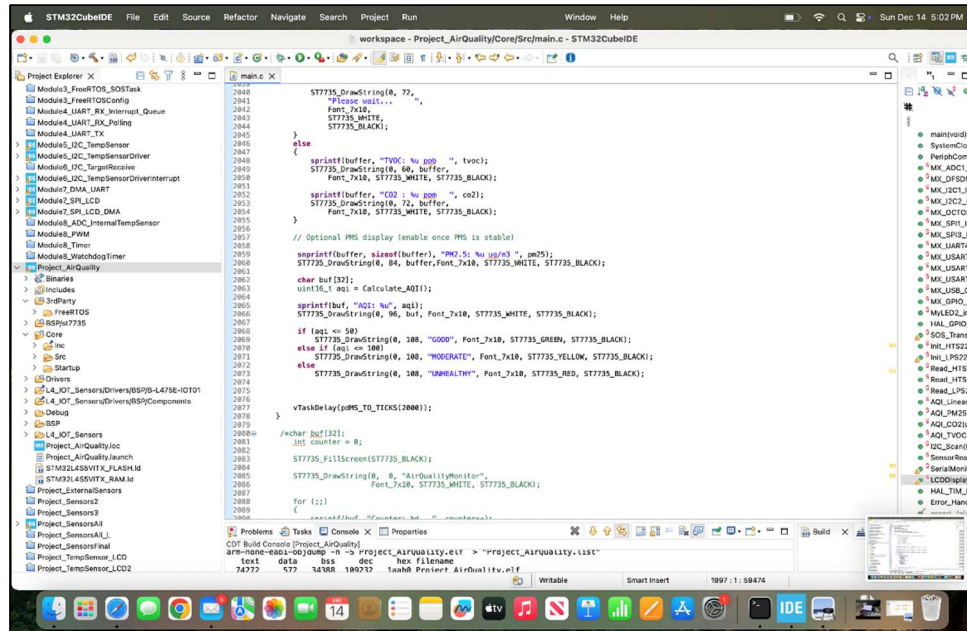


Figure 15:

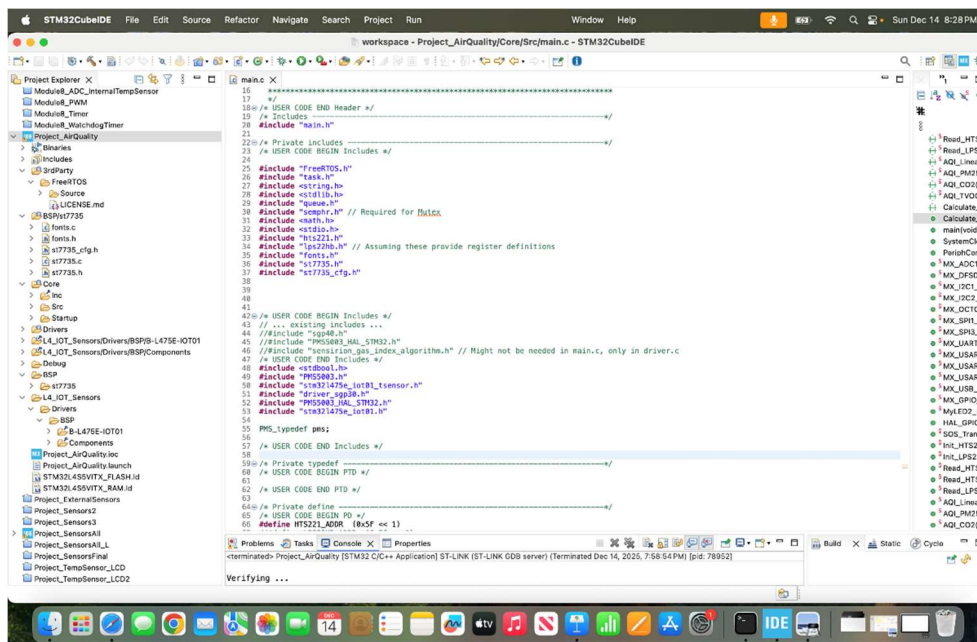


Figure 16:

Challenges Faced:

- **Hardware Interfacing:** This involved overcoming issues such as debugging signal integrity, pinning configurations for various I2C, UART sensors, and managing bus conflicts.
- **RTOS Synchronization:** One of the challenges is dealing with the data race issue by effectively using sensorDataMutex to synchronize accesses to global shared variables of the sensor by multiple tasks.
- **Sensor Reliability:** Validating data accuracy from low-cost sensors is a challenge, causing software to filter out implausible or erroneous data.
- **Data Interpretation:** Establishing standardized “UNHEALTHY,” “MODERATE” and “GOOD” levels for various air pollutants such as PM2.5, CO2, TVOC, with strict references to health-based criteria was one of the interpretive tasks.

VI. Testing and Validation:

The goal of testing is to confirm that hardware components communicate correctly, software logic is sound, and the final output accurately reflects environmental data.

1. Hardware Unit Testing

Connectivity: Confirm all sensor addresses are detected on the I2C bus using your I2C_Scan function.

Peripherals: Visually inspect that the LCD initializes correctly and the UART callback for the PMS5003 is receiving data bytes.

2. RTOS & Data Integrity Testing

Task Execution: Verify all tasks (SensorReadTask, LCD Display, etc.) are running periodically without blocking the system using the FreeRTOS task viewer (via a JTAG debugger).

Mutex Validation: Use the debugger to ensure that shared global variables are accessed only within the mutex-protected regions, preventing data race conditions.

3. System Functional Testing

Accuracy Check: Compare the displayed temperature, humidity, and pressure readings against a known, reliable external measuring device.

Status Logic: Manually input values to test the "UNHEALTHY," "MODERATE," and "GOOD" air quality logic (or expose the sensor to poor air conditions like a puff of breath for CO2 spike verification).

Reliability: Run the device continuously for an extended period (24+ hours) to ensure long-term stability and catch any memory leaks or unexpected crashes.

Project Results:

Output1:

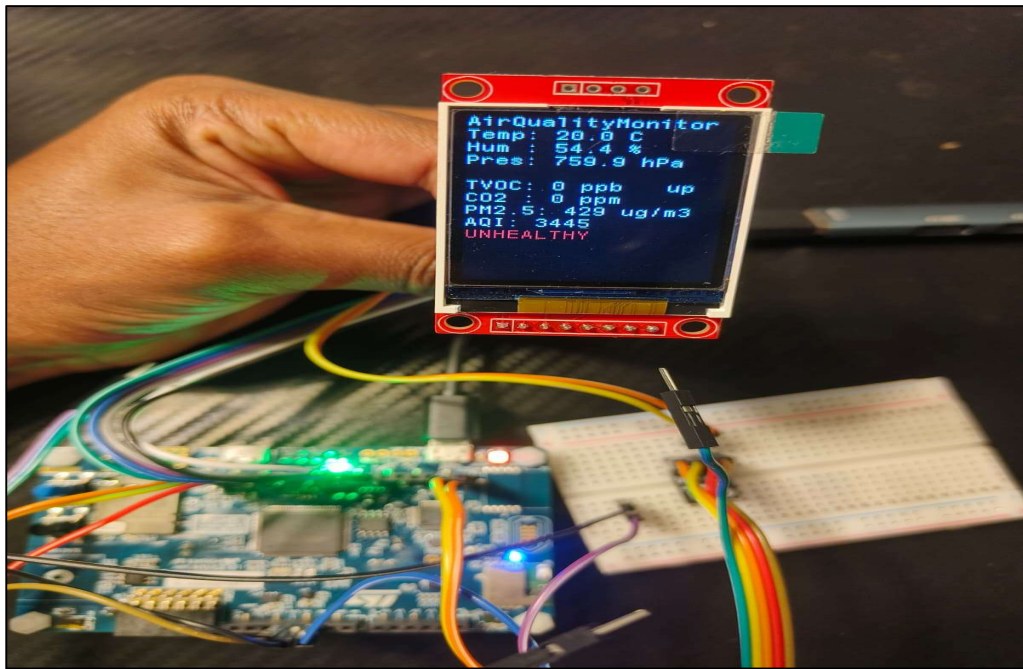


Figure 11: Output showing UNHEALTHY AQI

Output-2:

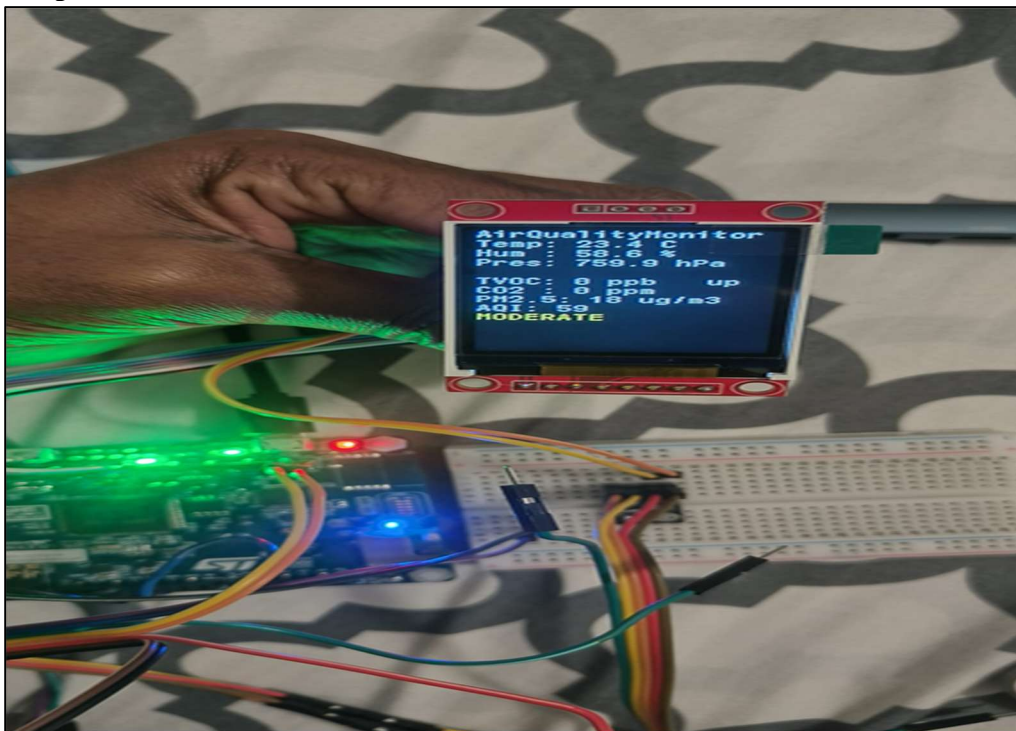


Figure 9: Output showing MODERATE AQI

Output-3:

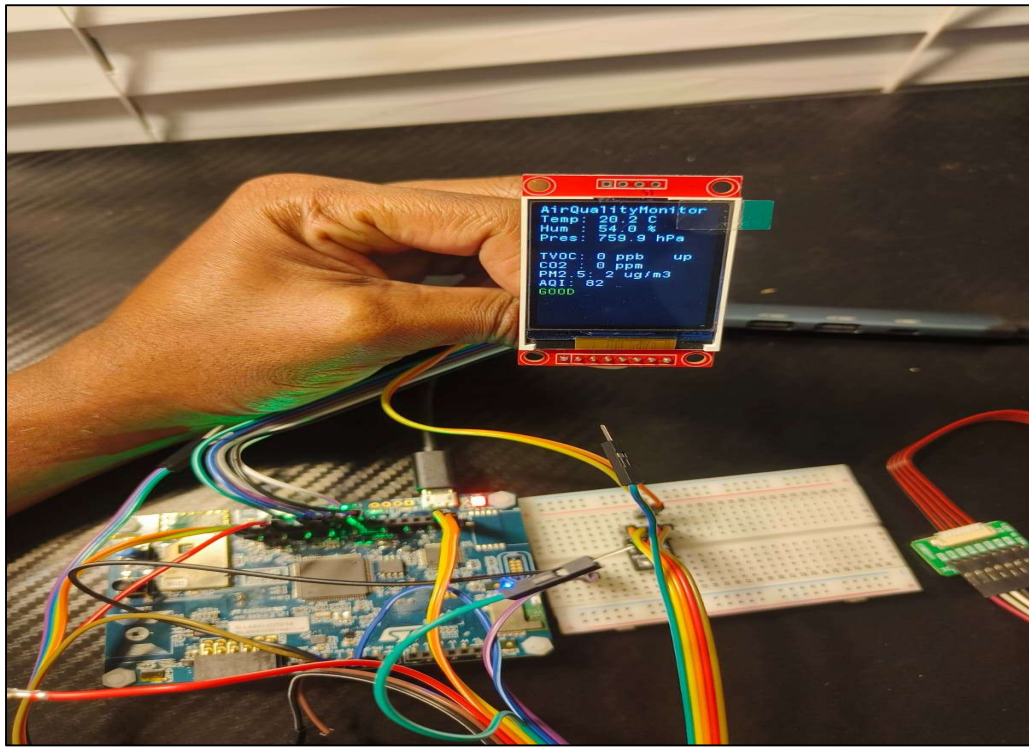


Figure 10: Output showing GOOD AQI

VIII. Future Enhancements:

- Averaging: Add data averaging to filter out sensor noises.
- Audible Alarm: Add a buzzer that sounds when air quality reaches “High” alert levels.
- Cloud Integration: Connect the device through Wi-Fi (the B-L4S5IOT01A development board contains a Wi-Fi module) with an IoT platform (AWS or Azure).
- BLE Mobile Application: Bluetooth Low Energy technology can be used to connect the monitor to a mobile application for local data analysis.
- Hardware Expansion
- Automated Control: Utilize a relay to automatically turn on an air purifier or ventilation fan if the levels of pollution are high.

IX. Conclusion:

Air Quality Monitor is an embedded system design that employs STM32 microcontrollers with the help of FreeRTOS for monitoring various environmental factors in real-time.

Key features and conclusion of outcomes include:

- Multi-Sensor Data Acquisition: Coupling of the HTS221, LPS22HB, SGP30, and PMS5003 Sensors using I2
- Efficient implementation of RTOS: application of tasks or mutexes of the FreeRTOS library (sensorDataMutex) for dealing with concurrency issues, data corruption issues, non-blocking systems, etc.
- Data Interpretation: The processing of raw data into standardized units for determining the air quality classification as “UNHEALTHY,” “MODERATE,” or “GOOD” relative to Air Quality guidelines.
- User Interface: Real-time data broadcast on an ST7735 SPI LCD screen for direct user interaction.
- It is evident that the final project accomplishes the development of a trustworthy solution for environmental monitoring using good principles of embedded systems development.